

Lab C：急件應提前多久送出？

問題：急件應提前多久送出，才不會為省能量而誤期？

急件在佇列

期限倒數
等待越久，越可能誤期

服務窗口 | gate

窗口開啟 → 可以送出
距限接近 → 優先處理
窗口關閉 → 不得送出

本頁讀法 | 先建立問題；下一頁開啟實際檔案，確認哪一行控制急件時間。

queue age | 等待時間

來源 *runner trace*；作用 顯示等待成本；單位 *s*；數字越高代表排隊越久。*deadline* 先判期限，*endpoint J* 只在服務結果後解釋。

定位唯一可編輯區塊：檔案、標記、目標值

先定位檔案與 marked block，再改唯一指定值。

檔案 | source

student_policy.py
/home/u24/lora-energy-lab/
本頁只定位，不執行

標記 | editable block

lab-c-batch-urgent
只允許區塊內的指定常數
順序與函式不動

目標 | one value

URGENT_MARGIN_S
20 → 5 → 30
BATCH_SIZE = 3 固定

不變範圍 | Lab A / B blocks、runner、scenario、schema、能量模型與其餘程式均維持原樣。

BATCH_SIZE | 批次數量門檻

來源 *student_policy.py* 的 Lab C marked block；單位 packet；作用 佇列至少 3 張才能使用 *FLUSH_BATCH*；本實驗固定為 3，不作為變更變數。

基準原始值：20 秒代表什麼？

基準值 20：距離期限還有 20 秒內，急件分支才會先回傳。

原始檔案： *student_policy.py* / Lab C marked block

```
# === LORA EDITABLE: lab-c-batch-urgent ===  
BATCH_SIZE = 3  
URGENT_MARGIN_S = 20  
# === LORA END EDITABLE: lab-c-batch-urgent ===  
if observation.urgent_pending and  
observation.urgent_due_in_s <= URGENT_MARGIN_S:  
    return SEND_URGENT
```

判斷順序

窗口關閉 → *SLEEP*
急件距限內 →
SEND_URGENT
間隔未到 → 休息動作
品質可送 → 批次或單筆
其餘 → *WAIT*

20 秒的意思 | 剩餘時間 $\leq$ 20 秒才進入急件分支；它優先於 *pacing* 與 *batch*，但仍受更前面的窗口判斷約束。

URGENT_MARGIN_S | 急件距限門檻

來源 Lab C marked block；單位 s；20 是原始門檻；值越小越晚出手，值越大越早出手。數值由 policy 影響 action timing，不直接寫入 endpoint J。

第一輪操作：基準 20，再執行候選 5

先建立基準，再完成唯一修改後執行候選；兩次都保存配對結果。

基準組 | baseline 20

Linux / macOS / WSL :
`bash course.sh run --lab C --case baseline`
 Windows PowerShell :
`.\course.cmd run --lab C --case baseline`
 Windows Command Prompt :
`course.cmd run --lab C --case baseline`

候選組 | candidate 5

完成 20 → 5 與 prediction lock 後：
 Linux / macOS / WSL :
`bash course.sh run --lab C --case candidate`
 Windows PowerShell :
`.\course.cmd run --lab C --case candidate`
 Windows Command Prompt :
`course.cmd run --lab C --case candidate`

收據與匯入 | 每次 stdout 印出新的 `result\_path`；同一 run 目錄保留 `result.json` + `endpoint-replay.json`，匯入時選 `result.json`。

服務先讀 | 欄位順序

先讀 `service_pass` / `deadline_pass`，再讀 `delivered_bits`，接著用 replay 的 state / packet 解釋 `endpoint_energy_j`；數值以本次 stdout path 為準，不猜檔名。

第一次修改：只把 20 改成 5

只改一行：20 → 5；程式碼頁先說清楚值的意義，不混入命令。

修改位置：student_policy.py / Lab C marked block

```
BATCH_SIZE = 3  
URGENT_MARGIN_S = 20
```

```
BATCH_SIZE = 3  
URGENT_MARGIN_S = 5
```

修改前：20 秒內才走 *SEND_URGENT*

修改後：5 秒內才走 *SEND_URGENT*

機制 | 門檻變小，急件分支要等到期限更近才返回；因此 normal pacing / batch 可能多等一段 delivery / deadline 需由結果驗證。

不變範圍

BATCH_SIZE = 3、Lab A / B blocks、runner、scenario、schema、其他常數 / 函式 / 檔案保持不變；這頁唯一高亮的是 *URGENT_MARGIN_S* 的新值 5。

候選執行前：先鎖定可反駁預測

先寫下可被資料推翻的預測：5 秒可能省電，也可能誤期。

距限 | due

urgent_due_in_s
單位 s；越小
越急

佇列 | queue

queue_size
單位 count；越
高壓力越大

服務／期限

service_pass /
deadline_pass
FAIL 先於 J

端點能量 | J

endpoint_energy_
j
單位 J；低只
代表耗能較少

預測句 | 較晚急件介入可能減少早期 wake / TX；delivery 或 deadline FAIL 即推翻純省能量說法。

證據順序

來源 observation / result summary / endpoint ledger；先讀服務／期限，再回到 packet 與 state 解釋 J。 *urgent_due_in_s* 越小越急； *endpoint_energy_j* 越低只代表耗能較少，不能覆蓋 FAIL。

候選結果怎麼讀：先匯入，再判服務

先判服務是否完成，再看端點用了多少能量。

基準組 | margin 20

服務／期限：PASS ／ PASS
送達封包：4 ／ 4 ；過期：0
端點能量：10.66 J

修改組 | margin 5

服務／期限：FAIL ／ FAIL
送達封包：3 ／ 4 ；過期：1
端點能量：9.74 J （少 0.92 J ）
低 J 仍不能稱成功

因果解讀 | candidate 少用 0.92 J 卻只送達 3 ／ 4 且兩個 gate FAIL ；低 J 不改寫為服務成功。

欄位讀法 | 來源與單位

service_pass ／ *deadline_pass* ：布林 gate ， PASS 才往下讀；*delivered_packets* ／ *expired_packets* ：packet count ，先判交付；*endpoint_energy_j* ：replay ／ ledger 的 J ，低值不能覆蓋 FAIL 。

第二次修改：只把 5 改成 30

只改一行：5 → 30；較早介入的理由與代價分開說明。

修改位置：student_policy.py / Lab C marked block

```
BATCH_SIZE = 3  
URGENT_MARGIN_S = 5
```

```
BATCH_SIZE = 3  
URGENT_MARGIN_S = 30
```

修改前：5 秒內才走 *SEND_URGENT*

修改後：30 秒內即可走
SEND_URGENT

機制與取捨 | 門檻變大，急件分支可較早返回；可能增加 `WAKE` / `TX`，也可能讓 required packet 在期限前完成。revision 只允許這一次。

不變範圍

BATCH_SIZE = 3、Lab A / B blocks、runner、scenario、schema、其他常數 / 函式 / 檔案保持不變；
這頁唯一高亮的是 *URGENT_MARGIN_S* 的新值 30。

修訂執行前：把較早介入寫成取捨預測

先比兩種介入時機，再看服務與能量取捨。

門檻 5 | 較晚介入

批次 → 期限 → 急件
較少早期 wake / TX，但 delivery
可能受壓

門檻 30 | 較早介入

急件 → wake / TX → delivery
較可能保住期限，也可能增加 energy
cost

條件式預測 | 只有 delivery、deadline、state ledger 與 endpoint J 同時對上，取捨敘述才成立。

證據欄位 | 來源與高低

delivery / deadline：result 的布林結果，PASS 才算完成；retry / expiry：packet events 的 count，越高表示重試／過期越多；endpoint J：ledger 的 J，越低耗能較少但不能覆蓋 FAIL。

第二輪操作：revision 30 freeze，再跑 surprise

先 freeze revision 30，再以同一份 policy 執行 surprise；兩步之間不再改碼。

修訂組 | revision 30

Linux / macOS / WSL：
`bash course.sh run --lab C --case revision --freeze`
 Windows PowerShell：
`.\course.cmd run --lab C --case revision --freeze`
 Windows Command Prompt：
`course.cmd run --lab C --case revision --freeze`

保留組 | surprise

沿用 frozen policy，沒有新 edit：
 Linux / macOS / WSL：
`bash course.sh run --lab C --case surprise`
 Windows PowerShell：
`.\course.cmd run --lab C --case surprise`
 Windows Command Prompt：
`course.cmd run --lab C --case surprise`

收據與限制 | 兩次 stdout 都提供新的 `result\_path`；revision 保留 freeze receipt，surprise 只讀 matching evidence，failure 只縮小 claim。

freeze | 凍結收據

來源 runner receipt；作用 綁定 policy bytes 與 case；判讀 identity 不符即停止。每個 run 的 result.json 與 endpoint-replay.json 必須成對。

20 / 5 / 30 : 沿同一條時間線讀三筆結果

20 / 5 / 30 沿同一條時間線比較：先分開機制，再讀觀測結果。

基準 | 20

機制：較早允許急件

觀測：PASS / PASS

4 / 4 ; expired 0 ; 10.66
J

候選 | 5

機制：較晚允許急件

觀測：FAIL / FAIL

3 / 4 ; expired 1 ; 9.74
J

修訂 | 30

機制：更早允許急件

觀測：PASS / PASS

4 / 4 ; expired 0 ; 10.66
J

20 |
primary

5 | late

30 |
early

Observed result | 5 少交付且兩個 gate FAIL ; 20 與 30 在固定 primary case 都在剩餘 10 秒時觸發，可能形成相同結果（null contrast）。

機制 ≠ 觀測 | 來源與判讀

J 的差異回到 action、WAKE / TX / RX、retry、delivery、expiry；數值取自 stdout result_path 配對檔。20 / 30 相同不代表 30 永遠最佳。

Surprise：沿用 30，保留失敗邊界

另一個情境沿用原程式；低端點能量不代表服務成功。

凍結修訂 | revision

$URGENT_MARGIN_S = 30$
程式內容不變，沿用原條件

保留情境 | same policy

服務／期限：FAIL／FAIL
送達：1／4；過期：3
retries：3；collisions：2
端點能量：5.41 J

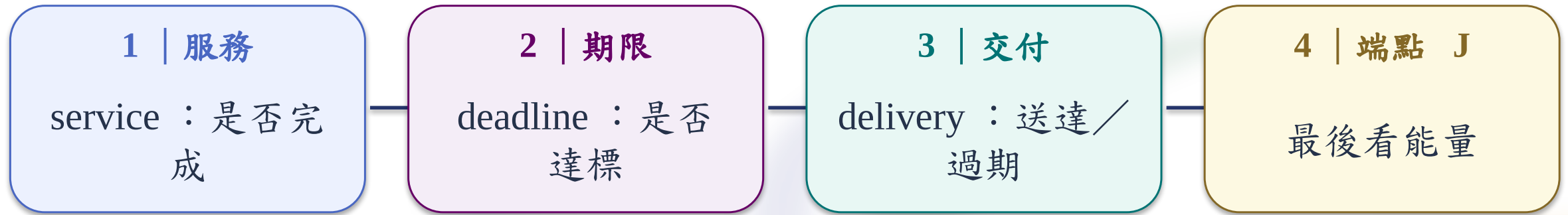
BOUNDARY READING | surprise 低 J 仍是 failure；保存為 frozen policy 的適用邊界。

來源範圍 | 數值判讀

來源 `result.json + endpoint-replay.json`；`delivered_packets` 是 1／4，`expired_packets` 是 3，`retries` 是 3，`collisions` 是 2；endpoint J 低仍不等於成功。

Lab C 結語：服務與期限先於端點 J

四道門依序檢查；最後才描述端點能量。



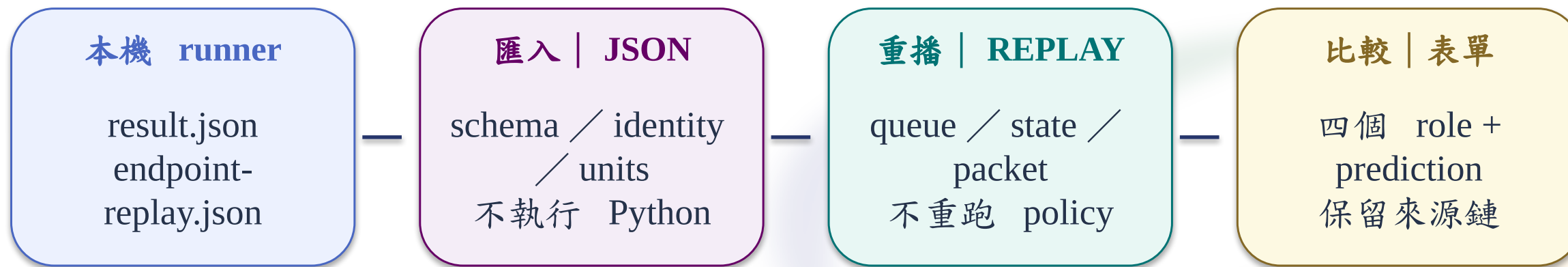
結論 | margin 5 低 J 但 gate FAIL ; margin 30 恢復固定條件 gate ; surprise 暴露窗口限制。

主張邊界

SIMULATED TEACHING DATA ; NOT LIVE ; NOT MEASURED ; NOT CANONICAL-PARITY-VERIFIED 。低 J 不覆蓋服務 FAIL 。

網站責任：匯入、重播、保存比較

本機先產生資料；網站只驗證、重播、保存比較。

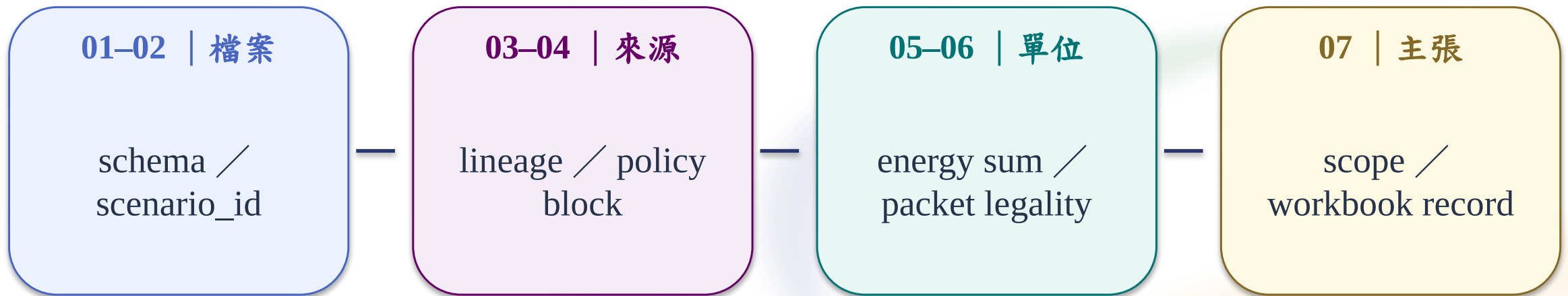


責任邊界 | 網站只接受 JSON artifact；Python policy 與 endpoint result 留在本機 runner。

網站讀法 | 現有 browser record
先讀 service / deadline，再讀 packet delivery / expiry，再讀 Radio / state event，最後讀 endpoint J
與 bit/J；browser accepted 不等同 live measurement，fresh runner parity 待取得。

匯入檢查：identity、units、事件逐層驗證

匯入檔案先逐層驗證；任何失敗都保留原狀。



FAIL CLOSED | 任一層失敗：session / workbook 不變；保存錯誤與 path，回 matching artifact。

scenario_id | 情境識別碼
來源 scenario package；值型態 identifier；作用 綁定同一情境；相同才可比較，缺失或不符即停止。
七層驗證完整後才 materialize replay。

事件重播：佇列 → 動作 → 狀態 → 封包 → J

畫面每一格都對應同一條事件記錄；一起讀動作與封包結果。

端點重播，選取畫面 1/56 · 0s

無線電 SLEEP	動作 —	佇列數 1	累積端點能量 J 0J
經過時間 0s	接觸識別碼 —	接觸狀態 關閉	連線品質 0

事件讀法

Queue / Action
佇列 → policy 動作

Radio / state
狀態：SLEEP / TX / RX

Packet / service
送達 / 過期 → 期限 / J

重播邊界 | 讀取已驗證 event ledger，不重新執行 policy，也不在瀏覽器推導新數值。

欄位讀法 | 來源、單位、高低

Queue：frame，count，越高待處理越多；Action：policy token，對回程式分支；Radio / state：frame event，確認 SLEEP / TX / RX；endpoint J：ledger，J，越低耗能較少，但服務 FAIL 仍先保留。

Workbook：保存四個角色的來源鏈

比較紀錄把四個角色放在同一條來源鏈，方便重開核對。

實驗/案例	顯示	角色	服務	端點能量 J	來源
A/baseline	目前	基準	FAIL	6.92 J	同情境備用資料

current browser record | execution ledger

四個角色

基準組
baseline 結果

候選組
candidate 修改

修訂組
revision gate

保留組
surprise 邊界

比較紀錄 | 保存 prediction、changed line、result_path、replay、gate、endpoint scope 與 recovery note。

provenance | 來源鏈

來源 /course workbook；值型態 saved record；作用 保存四個角色的 lineage；缺必要 evidence 就維持 INCOMPLETE，不補寫答案。

保存與重開：核對身份，不補造結果

保存、關閉、重開都先核對身份，再恢復同一條來源鏈。

01 | 保存 SAVE

保存預測＋結果／重播

02 | 關閉 CLOSE

不改 runner 檔案

03 | 重開

OPEN／REBIND：核對情境＋端點

完成度 | 身份、prediction、result／replay 齊全才 COMPLETE；缺件維持 INCOMPLETE 並 recovery，不改寫 FAIL。

COMPLETE／INCOMPLETE | 完成度狀態

來源 workbook validator；值型態 status enum；作用 反映必要 evidence 是否齊全；判讀 COMPLETE 不等於 service PASS，也不等於 scientific parity。